

grounded · cited · searchable

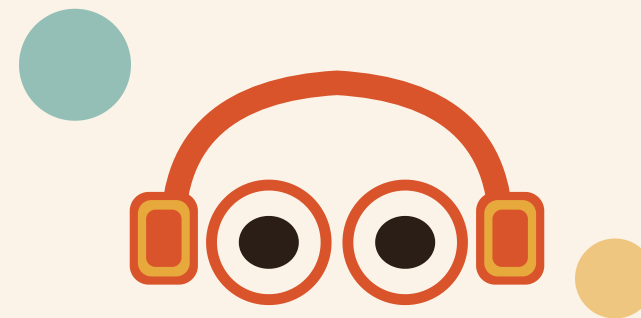
podcast *search.*

ask a question, get an answer grounded in real
podcast transcripts — with citations and
timestamps back to the moment.

🔍 ask in plain English

🗨️ answers that cite

🔌 MCP-ready



the problem

the insight is at minute 47.

podcasts are a goldmine of operator knowledge in a format that's hostile to retrieval — and an AI answer you can't *verify* is no better than a guess.

🕒 a two-hour episode

🔍 scrubbing back and forth

🔍 unsearchable, unquotable

how you find a quote today

vs

what we want

“what did they say about pricing?” → the exact quote, the episode, the timestamp.

the promise

ask in plain English. get
an answer you can act on.

🔍 how do they think about pricing?



Most operators price on value, not cost.

"We charge for the outcome we create, not the hours we put in."

🔊 My First Million

⌚ 47:12

every claim traces to the tape ↘

how it's built

small, boring, self-hosted parts.

no exotic infra — one self-hosted inference gateway serves speech, embeddings & chat behind a single endpoint; everything else is a couple of stores and a thin app.



transcribe

Parakeet · via gateway



embed + store

Pinecone · integrated embed



answer

Gemma 4 E4B



the server

FastAPI (Python)



the face

React, single-file SPA



deploy

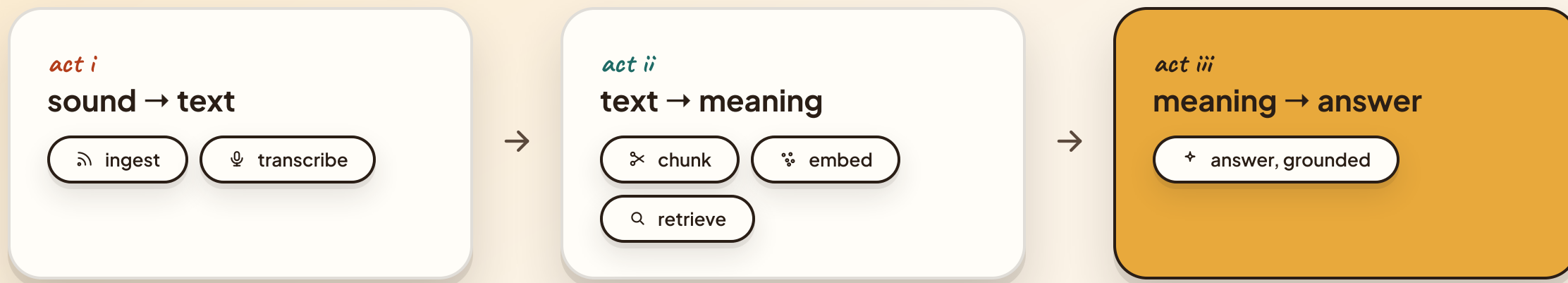
Docker → VPS

one shared **engine** drives two surfaces — the web app and an MCP server for agents.

the pipeline

one assembly line, in *three acts*

raw audio becomes a searchable, citable moment — by transforming it three times.



the data flows *one direction* — and trust is built in at every step.

read the *feeds*, find the audio

point it at a list of RSS feeds; it resolves each episode's real audio URL — even through an Apple Podcasts link.

My First Million

Acquired

Indie Hackers

Lenny's Podcast

Lex Fridman

SaaStr

Masters of Scale

How



19 shows wired in

operator & product podcasts, thousands of episodes

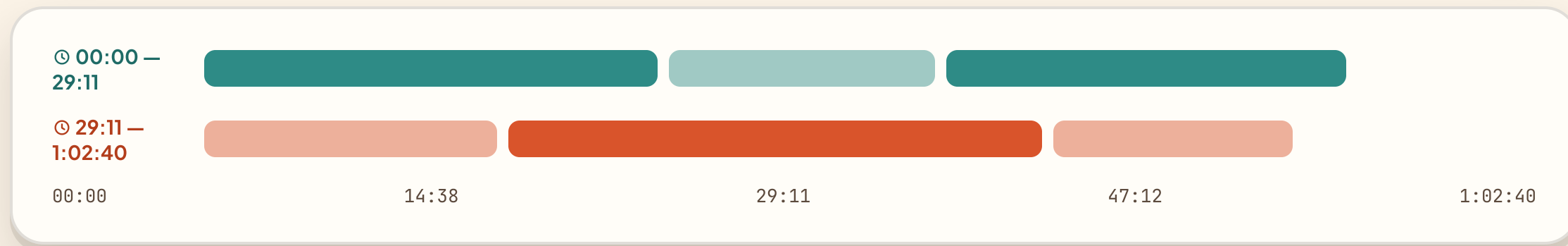


round-robin + dedup

interleave shows so no feed hogs the queue; skip audio already indexed

audio in, a timestamped transcript out

Parakeet, served by a self-hosted inference gateway, returns per-utterance text with timestamps — what was said, and exactly when.



🕒 utterance timestamps

📄 punctuated + formatted

⚡ async — submit a URL, poll for the result

📁 raw transcript saved to Postgres

✂ act ii · chunk

cut the transcript into bite-size passages.

a two-hour transcript

hundreds of utterances · tangents, ad reads,
cross-talk... one long, unsearchable wall of text.



group $\approx$ 1,000 chars

big enough to hold a
thought, small enough to
pinpoint

CHUNK · c0007

“...we charge for the outcome, not the hours...”

🕒 47:12 → 48:30

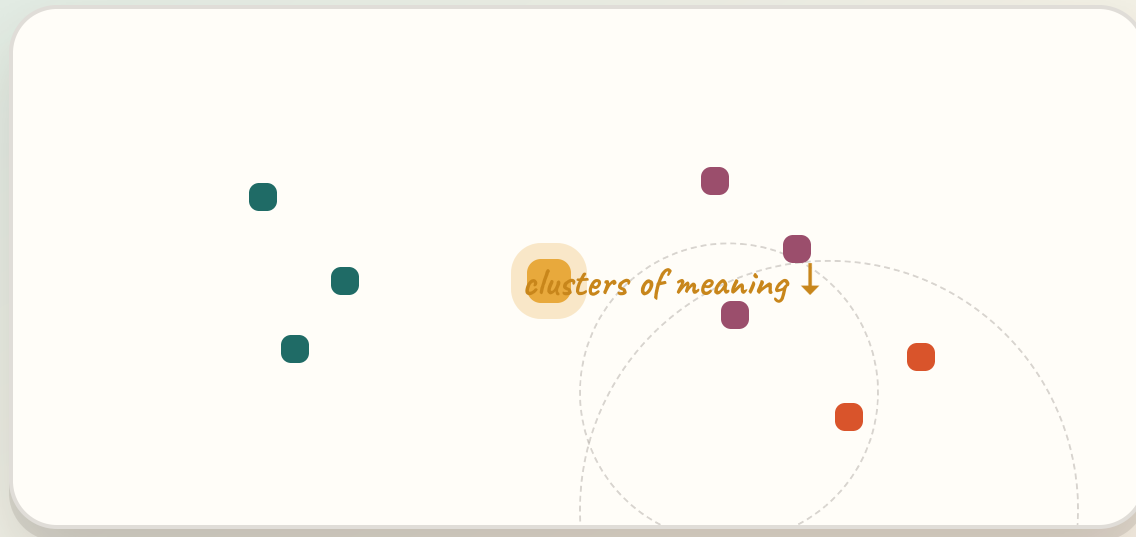
🎧 My First Million

every chunk keeps its timestamps — so any answer can point straight back to the moment in the audio.

act ii · embed & store

turn each passage into a *point in space*

every chunk becomes a 768-dim vector. passages about the same idea
land near each other — even with no shared words.



Pinecone integrated embedding

llama-text-embed-v2 — Pinecone embeds the text on
upsert, no separate step



stored in Pinecone

text + metadata: episode, show, timestamp



metadata travels with it

so every match stays traceable to a moment

🔍 act ii · retrieve

find the nearest moments — then re-rank.

Pinecone embeds the query and returns the closest 25 chunks. then a keyword boost rescues the exact names, and a dynamic threshold trims to the keepers.



① broad vector pull

top_k = 25 by cosine similarity



② keyword re-rank

overlap bonus, up to +0.15



③ dynamic threshold

keep within 55% of the best score

*why hybrid? vectors get the **vibe**, keywords get the **name**.*

VECTOR ONLY

nails “pricing strategy” ... but buries the chunk that names “RevenueCat”.

+ KEYWORD BOOST

lifts proper nouns & exact phrases back to the top — most of a re-ranker's win, almost none of the cost.

nothing clears the bar? it says so — it won't force an answer out of weak matches.

write the answer from the excerpts — and only the excerpts.

the retrieved chunks (with their timestamps) go to Gemma 4 E4B under a strict system prompt. it synthesizes; it doesn't recall.



cite every claim

name the episode, quote the words



no outside knowledge

never the model's training data



admit the gaps

"I don't see that discussed"

system prompt

Every claim you make **MUST** be traceable to a specific excerpt provided below.
You are a **retrieval system**, not a knowledge base.

the one idea that matters



grounded, not generative.

the LLM is a synthesizer, not a source. the system prompt forces **every claim back to a real excerpt** — which is what makes the answers trustworthy enough to act on.

no confident guessing. if it isn't in the tape, it isn't in the answer.

🕒 *the payoff · citations & timestamps*

every answer is a receipt.

alongside the prose, the answer carries its sources — ranked by relevance, each linking to the exact moment in the episode.



My First Million — “Pricing for value”

🔄 score 0.71

00:00



47:12



jump to the moment

timestamp → the audio at 47:12



the exact words

a verbatim quote, in context



best chunk per show

deduped, ranked by score

the same library, open to *any agent*.

one engine, two front doors. the same retrieval is exposed over the Model Context Protocol — so Claude Code, Claude Desktop or Cursor can search the transcripts directly.



search_podcasts
semantic search, all shows



list_episodes
everything indexed



get_episode_transcript
full text, partial match



search_in_episode
scoped to one show



find_related_episodes
who discusses it most



get_library_stats
episodes, chunks, detail

the MCP server returns raw context — it never calls an LLM. the calling agent reads, then writes its own grounded answer.

 *not a toy*

built like production software.

crawling the open web is messy and the catalog is read constantly — so it's engineered to feel instant, survive failure, and just run.



fast

- catalog cached in memory, PostgreSQL-backed
- stale-while-revalidate on a 2-min TTL
- one shared engine behind app + MCP



sturdy

- auth + rate limit (60/min)
- audio-URL dedup; skip a show after 3 fails
- oversize audio compressed to fit the 50 MB cap



shipped

- Docker on a small ~\$5/mo VPS
- heavy lifting lives behind the gateway
- auto-deployed from main, health-checked

every failure is logged with a reason and streamed to the UI — watch the crawl, and know why something was skipped.

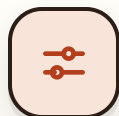
what I'd tell you

retrieval is the product. the model is the garnish.

a two-hour episode becomes ~6 cited sentences you can trust — that's the whole game.



ground every claim. cite the excerpt, or don't say it.



hybrid beats pure vector. a tiny keyword boost rescues names & numbers.



metadata is the magic. timestamps turn a match into a receipt.



**press play on
your back catalog.**

podsearch.space

[open-source · MIT](#)